100

Chapter 1: A Day in the Village 116

110

The sun rises over the hills. Birds sing in the cool air as the village wakes. Farmers go to the fields, and children walk to school. 112

The sky glows bright and warm. People rest after work, and friends share tea and laughter in the small café. 112

The sun sets, and lights appear in the windows. Families gather for dinner, and soon the village falls quiet again. 112

114

Chapter 2: A Storm On the Horizon

A farmer is awoken by thunder. He looks out the window and sees a storm coming. He tries to remember if he has secured his tools and equipment. . . .

114

FIG. 1

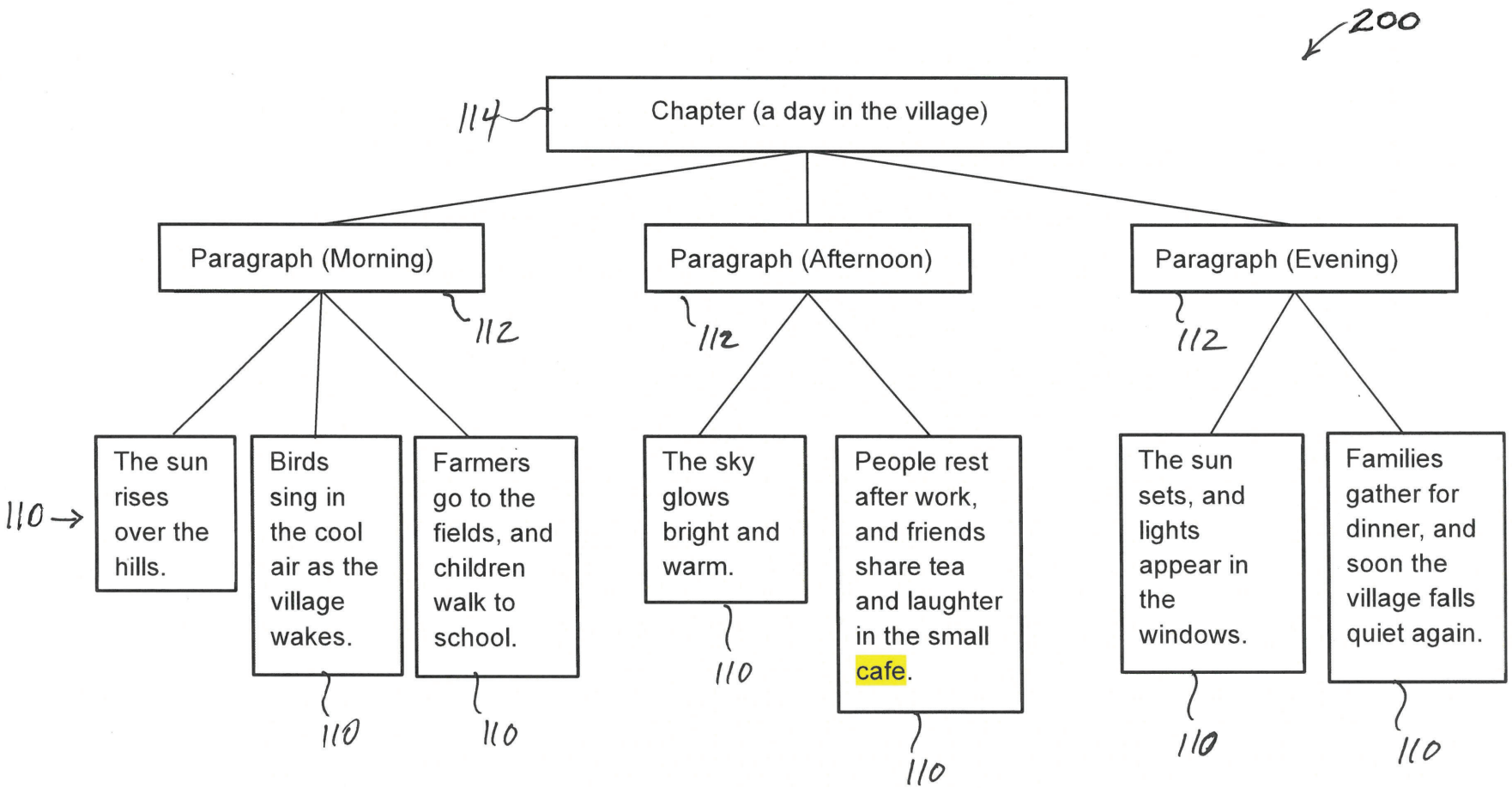
200
114
Chapter (a day in the village)
Paragraph (Morning)
Paragraph (Afternoon)
Paragraph (Evening)
112
112
112
110
The sun rises over the hills.
Birds sing in the cool air as the village wakes.
Farmers go to the fields, and children walk to school.
The sky glows bright and warm.
People rest after work, and friends share tea and laughter in the small cafe.
The sun sets, and lights appear in the windows.
Families gather for dinner, and soon the village falls quiet again.
110
110
110
110
110
110
110

FIG. 2

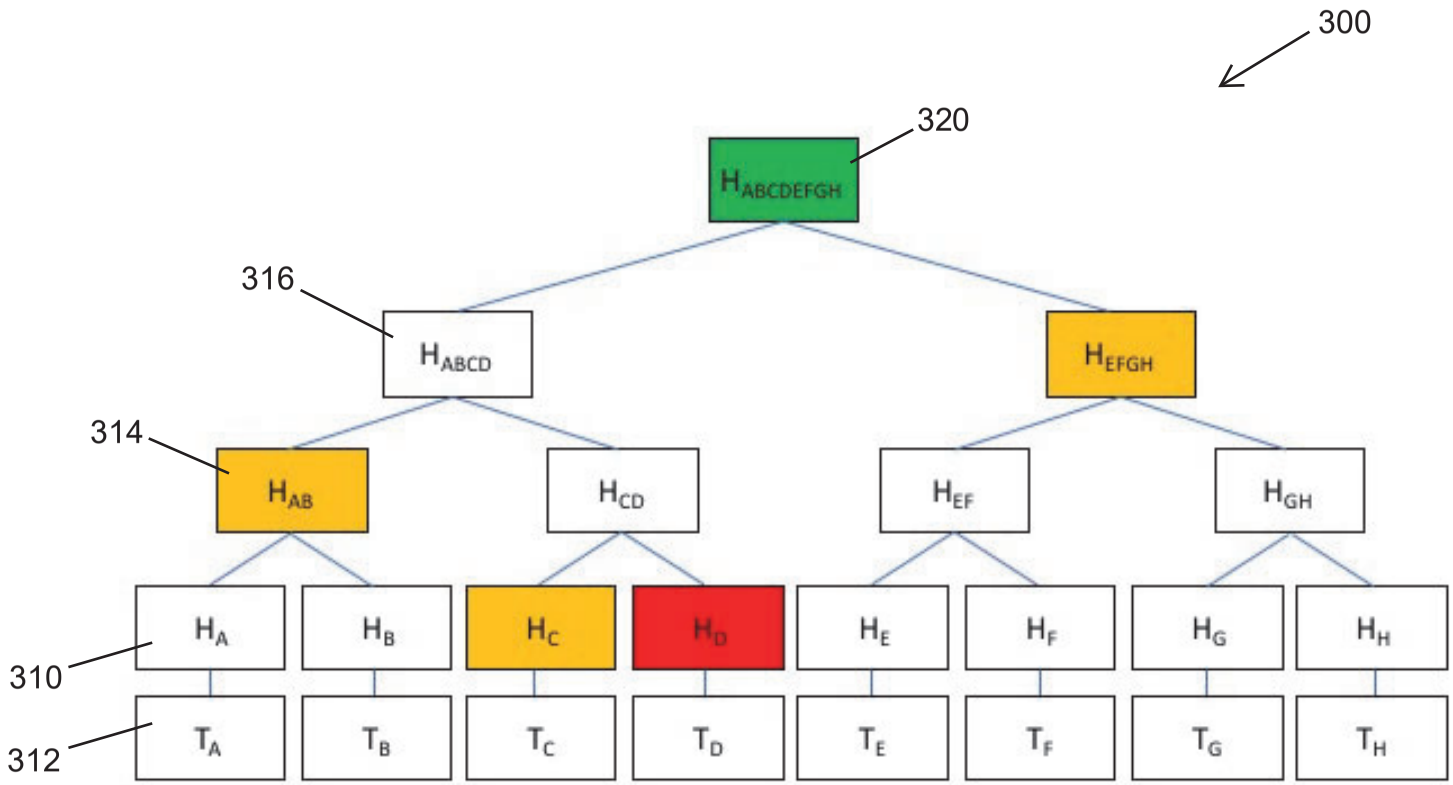
300
320
$H_{ABCDEFGH}$
316
H_{ABCD}
H_{EFGH}
314
H_{AB}
H_{CD}
H_{EF}
H_{GH}
H_A
H_B
H_C
H_D
H_E
H_F
H_G
H_H
310
312
T_A
T_B
T_C
T_D
T_E
T_F
T_G
T_H

FIG. 3

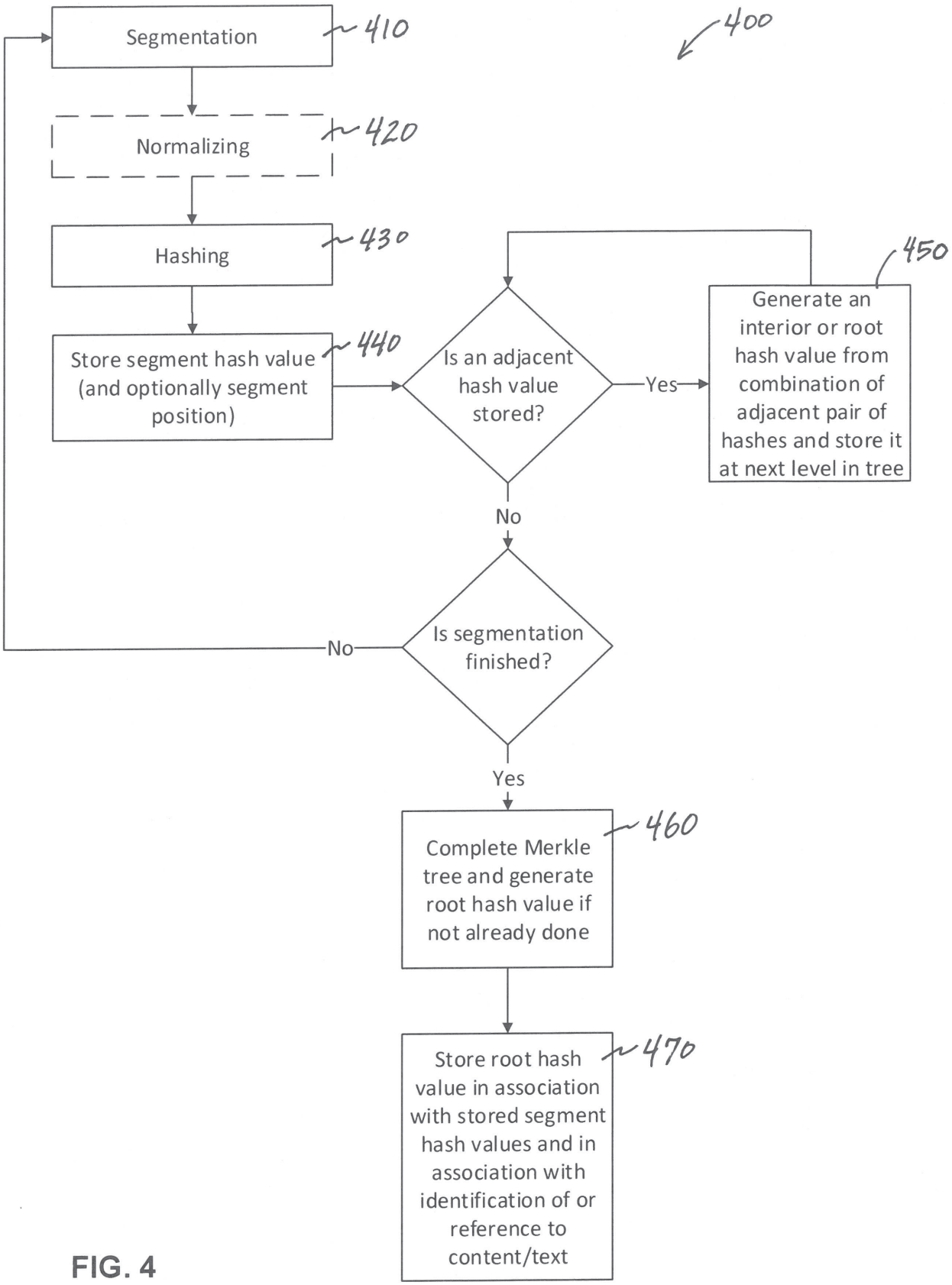
400
Segmentation
410
Normalizing
420
Hashing
430
Store segment hash value (and optionally segment position)
440
Is an adjacent hash value stored?
Yes
Generate an interior or root hash value from combination of adjacent pair of hashes and store it at next level in tree
450
No
Is segmentation finished?
No
Yes
Complete Merkle tree and generate root hash value if not already done
460
Store root hash value in association with stored segment hash values and in association with identification of or reference to content/text
470

FIG. 4

Streaming Merkle Tree Construction with Buffer Management

Step 501	**Receive & Hash First Segment** Buffer accumulates T_A until segment boundary. Hash generated and stored.	**HASH BUFFER (MAX 4)** H_A – – –	**STREAMED OUTPUT** "The sun rises over the hills."
Step 502	**Second Segment → Combine** H_B generated. Adjacent pair: $H_A + H_B \rightarrow H_{AB}$.	**HASH BUFFER** H_{AB} – – –	**STREAMED OUTPUT** "Birds sing in the cool air..."
Step 503	**Third Segment** H_C generated. No adjacent unpaired hash. Store alongside H_{AB}.	**HASH BUFFER** H_{AB} H_C – –	**STREAMED OUTPUT** "Farmers go to the fields..."
Step 504	**Fourth Segment → Cascade** $H_D \rightarrow H_{CD} \rightarrow H_{ABCD}$. Two combines cascade up.	**HASH BUFFER** H_{ABCD} – – –	**STREAMED OUTPUT** "The sky glows bright and warm."
Steps 505-507	**Build Right Subtree** H_E, H_F, H_G generated. H_{EF} combined.	**HASH BUFFER** H_{ABCD} H_{EF} H_G –	**STREAMED OUTPUT** "People rest..." / "Sun sets..." / "Families..."
Step 508	**Close Stream → Root** Pad H_H=0 → H_{GH} → H_{EFGH} → **Root $H_{ABCDEFGH}$**	**FINAL MERKLE ROOT** $H_{ABCDEFGH}$	**FINAL OUTPUT** [Manifest embedded via Unicode variation selectors]

Empty Slot | Newly Generated | Retained Hash | Combined Interior | Merkle Root

FIG. 5

Streaming Merkle Tree Construction Sequence with Bounded Memory Usage

Resulting Merkle Tree Structure

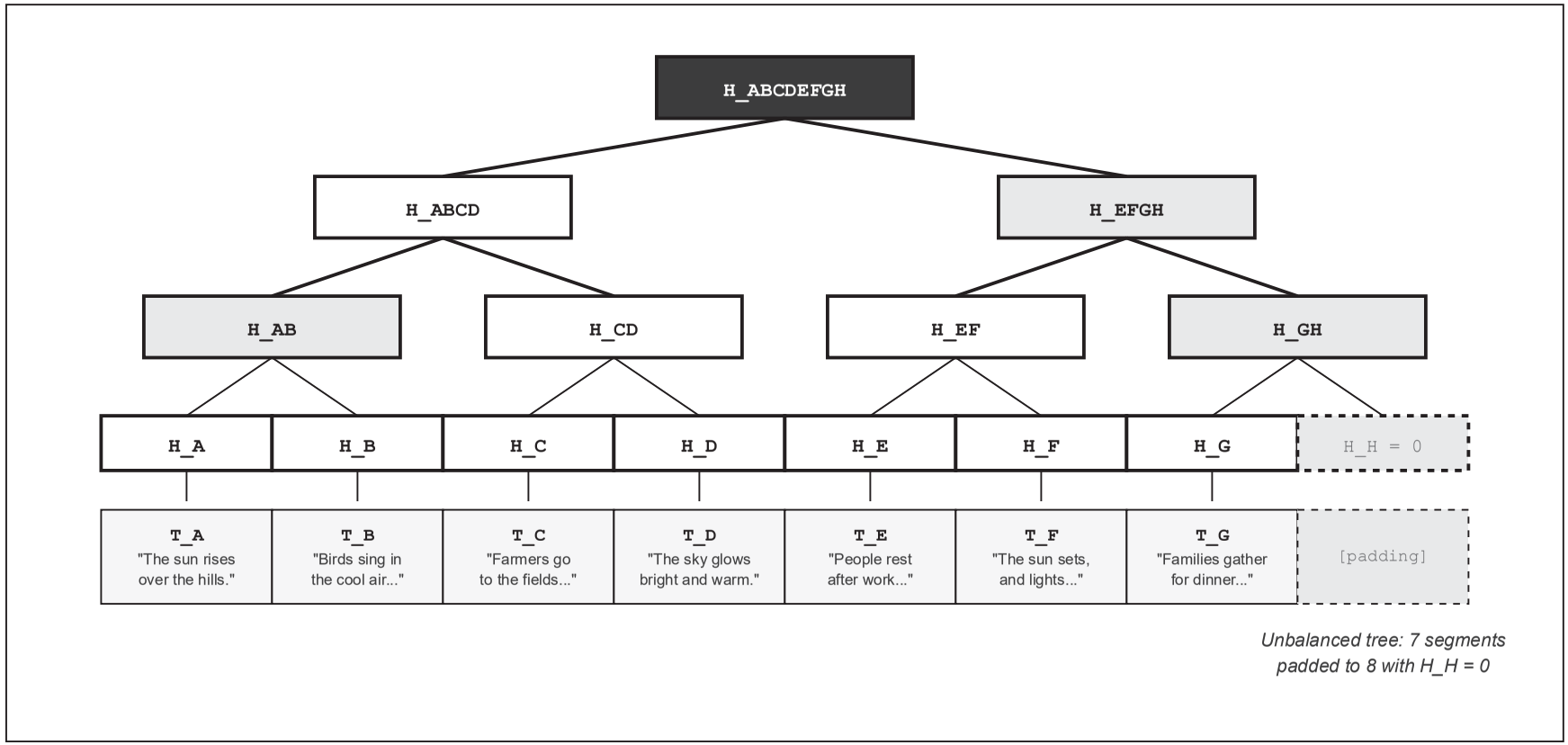

FIG. 5B

Resulting Merkle Tree Structure from Streaming Construction

System Architecture Overview

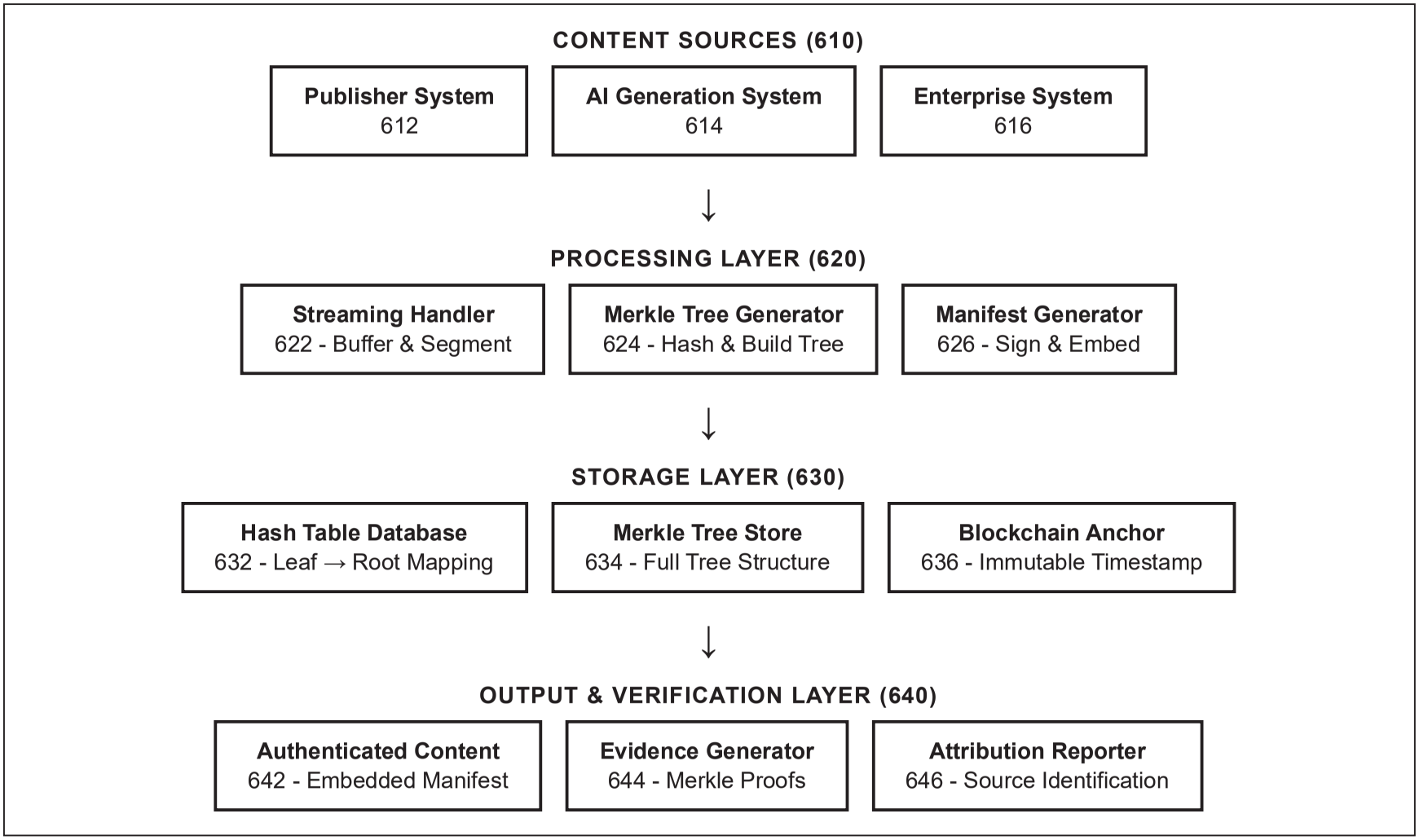

FIG. 6

System architecture for granular content authentication and attribution

Document Segmentation with Hierarchical Structure

Document 700: "A Day in the Village"

Paragraph P_1 (Morning) - 710

"The sun rises over the hills." T_A "Birds sing in the cool air as the village wakes." T_B "Farmers go to the fields, and children walk to school." T_C

Paragraph P_2 (Afternoon) - 720

"The sky glows bright and warm." T_D "People rest after work, and friends share tea and laughter in the small cafe." T_E

Paragraph P_3 (Evening) - 730

"The sun sets, and lights appear in the windows." T_F "Families gather for dinner, and soon the village falls quiet again." T_G

T_A Sentence 1 T_B Sentence 2 T_C Sentence 3 T_D Sentence 4 T_E Sentence 5 T_F Sentence 6

T_G Sentence 7

FIG. 7

Text document segmented into hierarchical units (paragraphs and sentences)

Hash Table Structure with Linked Lists for Multiple Sources

Leaf Hash → Merkle Root(s) Mapping (810)

H_A	Root_Doc1 → *NULL*
H_B	Root_Doc1 → *NULL*
H_X	Root_Doc2 → Root_Doc5 → Root_Doc8 → *NULL*
H_Y	Root_Doc3 → *NULL*
...	...

Merkle Root → Document Metadata (820)

Root_Doc1	doc_id: "village_story" author: "Publisher A" timestamp: 2025-12-01
Root_Doc2	doc_id: "news_article" author: "Publisher B" timestamp: 2025-11-15
Root_Doc5	doc_id: "blog_post" author: "Publisher C" timestamp: 2025-10-20
...	...

Note: H_X appears in multiple documents (Doc2, Doc5, Doc8) - linked list preserves all source attributions. Order may reflect chronology (earliest first) or prominence (most authoritative first).

FIG. 8

Database hash table structure with linked lists for segments appearing in multiple source documents

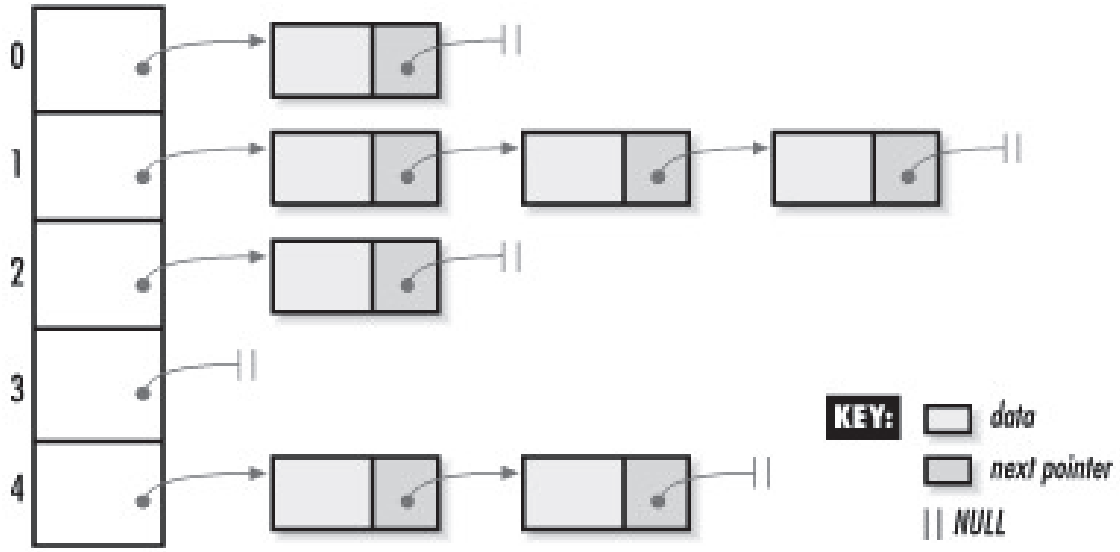

FIG. 8B

Full C2PA Manifest Structure vs. Lightweight UUID Manifest

Full C2PA Manifest (910)

Header (912)

Magic	C2PATXT\0
Version	1
Length	2048 bytes

Payload - JUMBF Container (914)

Content Hash	SHA256(text)
Merkle Root	H_ABCDEFGH
Segment Count	7
Assertions	[author, date, ...]
Claim Generator	Encypher/1.0
Merkle Tree	[optional full tree]

Signature Block (916)

Algorithm	ES256
Certificate	[X.509 cert]
Signature	[digital sig]

~2-10 KB embedded via Unicode variation selectors

Lightweight UUID Manifest (920)

Prefix (922)	UUID (924)	Assertion (926)
U+FEFF BOM marker	550e8400-e29b-... 128-bit identifier	[compact data] Optional inline

↓

External Repository (928)

UUID	550e8400-e29b-...
Content Hash	SHA256(text)
Merkle Root	H_ABCDEFGH
Full Merkle Tree	[all nodes]
Full Assertions	[complete data]

~50-200 bytes embedded; full data in external repository

FIG. 9

Comparison of full C2PA manifest embedding vs. lightweight UUID pointer to external repository

Dual-Binding Manifest Construction Sequence

Step 1001

Hash the Text Content

```
H_text = SHA256(normalize(text))
```

Step 1002

Create Preliminary Manifest with Placeholder

```
preliminary_manifest = {
  content_hash: H_text,
  merkle_root: H_ABCDEFGH,
  assertions: [...],
  self_hash: PLACEHOLDER
}
```

Step 1003

Hash the Preliminary Manifest

```
H_preliminary = SHA256(preliminary_manifest)
```

Step 1004

Replace Placeholder → Create Final Manifest

```
final_manifest = {
  content_hash: H_text,
  merkle_root: H_ABCDEFGH,
  assertions: [...],
  self_hash: H_preliminary
}
```

Step 1005

Digitally Sign the Final Manifest

```
signed_manifest = sign(final_manifest, private_key)
```

Step 1006

Embed Signed Manifest in Text

```
authenticated_text = text + encode_unicode(signed_manifest)
```

Dual Binding: The manifest binds to both (1) the text content via H_text, and (2) itself via H_preliminary. This ensures any modification to either the text OR the manifest after signing will be detected during verification.

FIG. 10

Dual-binding manifest construction method for tamper-evident content authentication

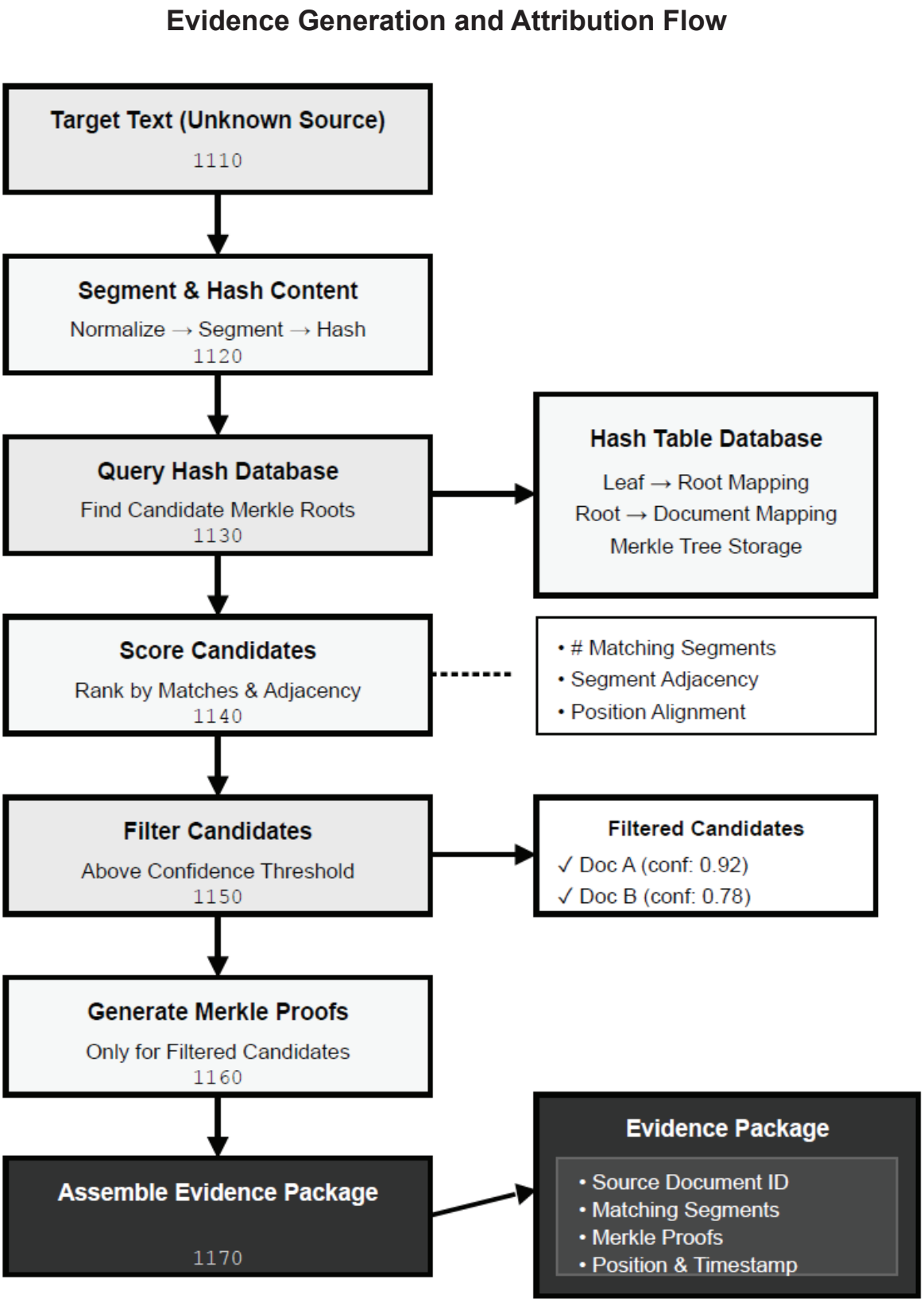

FIG. 11

Evidence generation flow from unknown target text to attributed source documents with Merkle proofs

Per-Document Revocation System

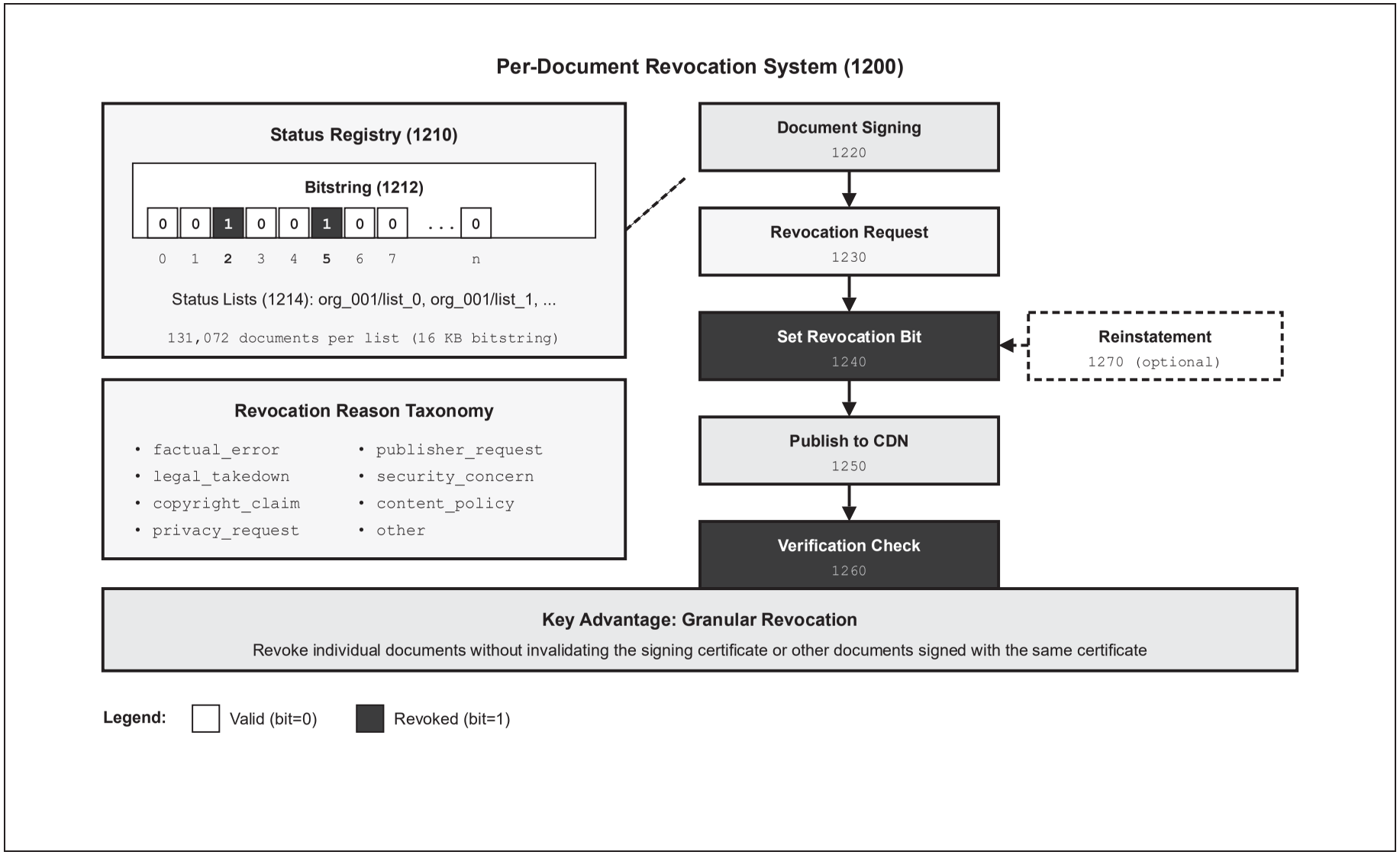

FIG. 12

Per-document revocation system with bitstring status registry enabling granular revocation without certificate invalidation

Distributed Embedding with Carrier Selection

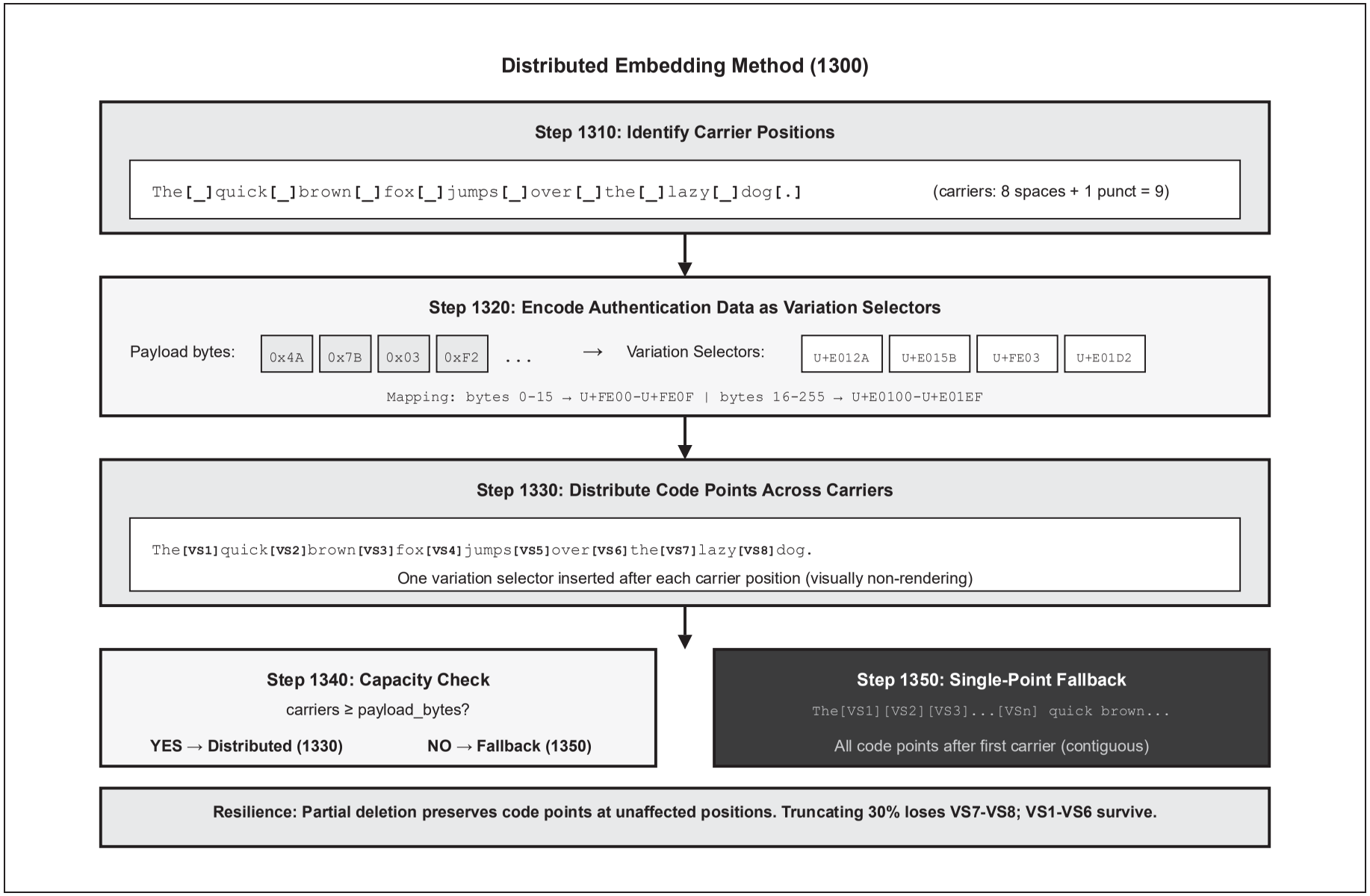

FIG. 13

Distributed embedding method with carrier selection for resilient content authentication

Two-Layer Embedding Architecture

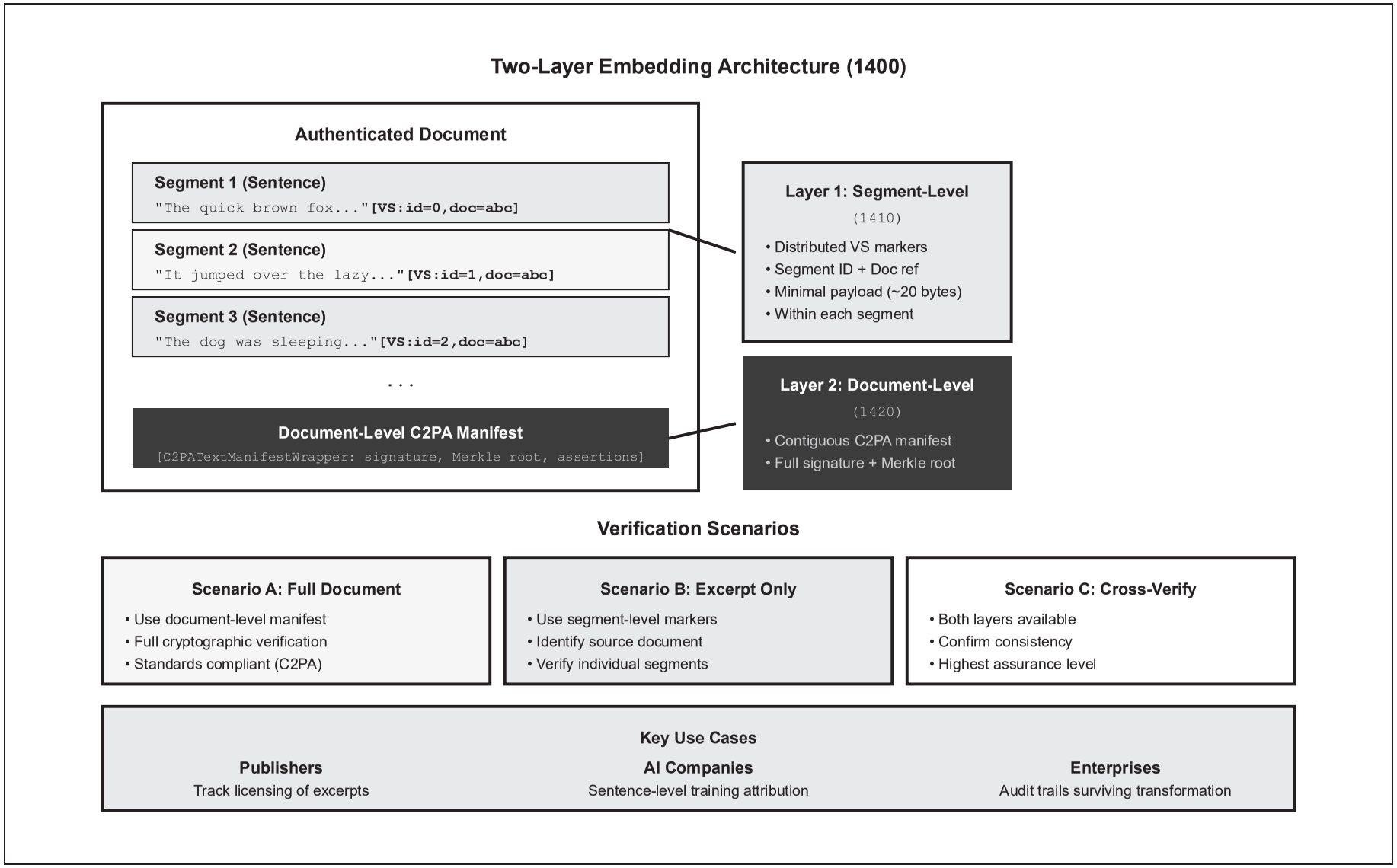

FIG. 14

Two-layer embedding architecture with segment-level and document-level authentication for flexible verification

Robust Fingerprinting for Semantic Similarity

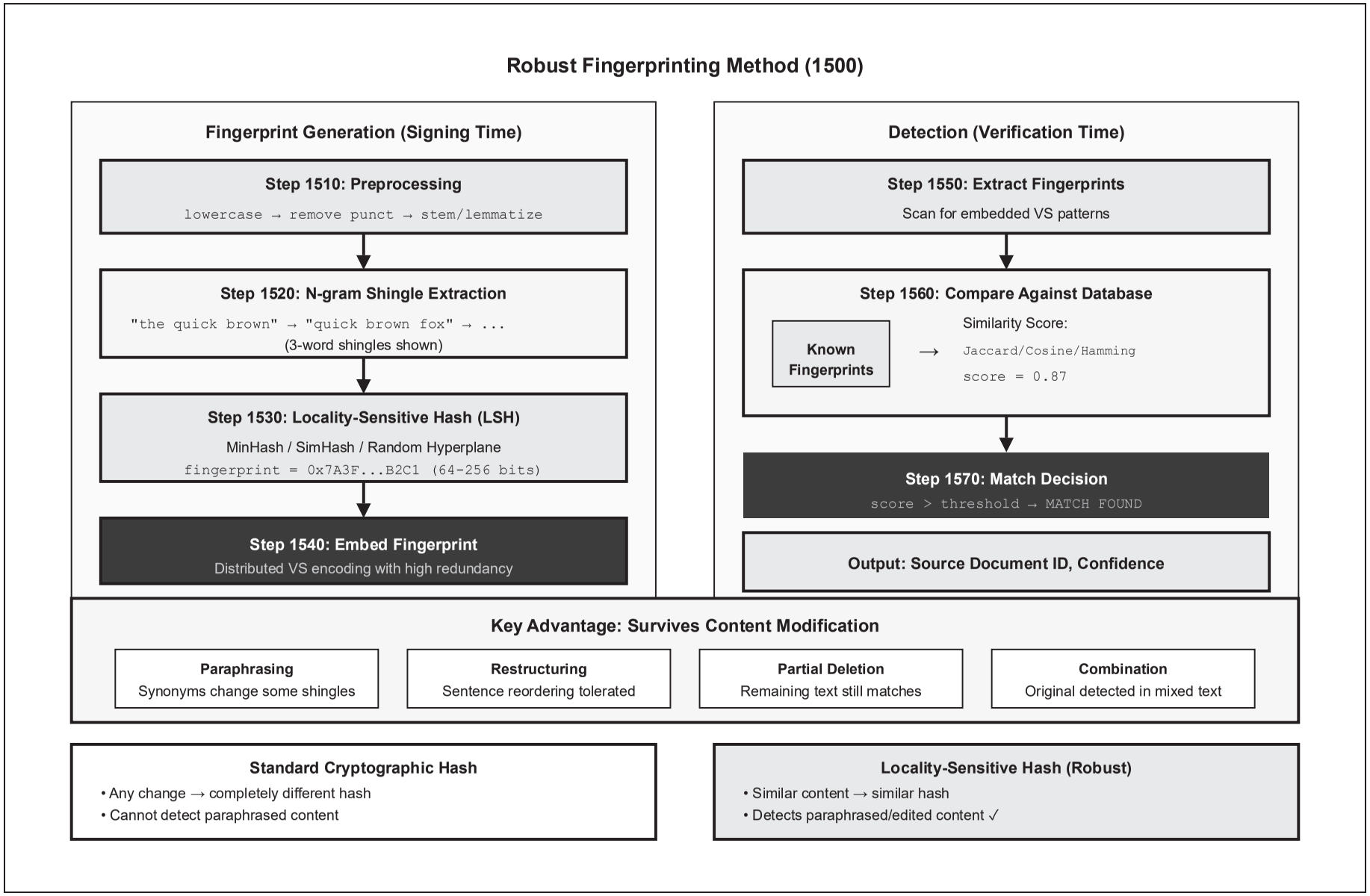

FIG. 15

Robust fingerprinting method using locality-sensitive hashing for semantic similarity detection